

TollRoad — Unit Economics (All-In AWS Cost to Stream)

What does it actually cost to stream one minute of music on TollRoad? This is a bottom-up marginal cost across **every AWS service in the path**, expressed per **minute**, per **song**, and per **user**. All rates are us-east-1 on-demand, **verified 2026-06-12 against the live AWS Pricing API** ([aws pricing get-products](#) — exact queries in the appendix). Marginal = the incremental cost of one more minute (free tiers are a fixed subsidy, excluded from the per-unit rate; treated separately at the end).

The canonical streamed minute

One minute of playback consumes a fixed, known basket of AWS resources:

Dimension	Value	Why
Audio bitrate	160 kbps → 1.2 MB/min	Music-quality HLS; bitrate is the dominant lever (see below)
CloudFront requests	~10 / min	HLS segments at ~6s each + manifest
/api/renew calls	~1.33 / min	60s signed cookie renewed every ~45s
DynamoDB write units	~4 / min	each renew = balance decrement (1 WRU) + metered-event put (1 WRU) + GS11 replication of the event (1 WRU, ALL projection) $\approx 3 \text{ WRU} \times 1.33 \text{ renews}$
DynamoDB stream reads	\$0	GetRecords calls made by a Lambda event-source mapping are not billed
Aurora DSQL ops	1 ledger insert + 1 summary upsert	per <i>minute</i> , not per renew — the rollup dedupes on user#track#minute
Lambda	0.01–1 invocation/min	batch ≤ 100 ; full batches at scale, ~per-event when traffic is sparse (cost is noise either way, see table)
KMS	~0 calls/min	SSE-KMS + CloudFront cache → no per-play decrypt (<i>requires S3 Bucket Keys — see caveats</i>)

Cost per minute streamed — every AWS service

Shown as **\$ per 1,000 minutes** (the per-minute figures are fractions of a cent, so 1,000 min is the readable anchor).

AWS service	Unit math	\$ / 1,000 min	% of total
CloudFront — egress	1.2 GB × \$0.085/GB	0.10200	88.3%
CloudFront — requests	10,000 req × \$0.01/10k	0.01000	8.7%
DynamoDB — writes	4,000 WRU × \$0.625/M	0.00250	2.16%
DynamoDB — streams	Lambda consumer → GetRecords free	0.00000	0%
DynamoDB — storage (event TTL)	0.3 KB/event, ~30-day resident	0.00008	0.07%
DynamoDB — reads (live meter)	0.5 RRU × \$0.125/M	0.00006	0.05%
Aurora DSQL — compute (DPU)	~0.073 DPU/min × \$8/M (<i>modeled</i>)	0.00058	0.50%
Aurora DSQL — storage (ledger)	100 B/min × \$0.33/GB-mo	0.00003	0.03%
KMS — key (amortized)	\$1/mo ÷ 8.17M min	0.00012	0.10%
S3 — origin GET + storage (amortized)	cache-miss reads only	0.00010	0.09%
Lambda — rollup	~10 req + ~0.4 GB-s (full batches)	0.00001	0.01%
TOTAL		0.11548	100%

All-in marginal cost ≈ \$0.0001155 per minute ≈ 0.01155¢ / minute.

Where the money actually goes

- **CloudFront (egress + requests) = ~97%** of the marginal cost. TollRoad is, economically, a **bandwidth business** wearing a database's clothes.
- **The entire data + compute plane — DynamoDB, Aurora DSQL, Lambda, KMS, S3 — is ~3%** combined (~\$0.00348 per 1,000 min). Per stream, **the databases are effectively free**; serverless + scale-to-zero means you pay for work done, and the work per minute is tiny.

Rolled up: per song, per user

Unit	Minutes	All-in AWS cost
Per minute	1	\$0.0001155 (0.0115¢)
Per song	3.0	\$0.000346 (0.035¢)
Per user / month	817	\$0.0944 (9.4¢)
Per user / year	9,800	\$1.132

So a fully-active listener — someone who streams as much as the average Spotify user — costs TollRoad about **\$1.13/year, all-in, across the entire AWS stack.**

Monthly AWS bill — the stack as defined in `infra/`

The marginal numbers above, re-expressed as the actual monthly bill the deployed stack (`infra/lib/tollroad-stack.ts`) would generate at three scales. Assumes a 10,000-track catalog (~36 GB at 160 kbps) and the always-free tiers (1 TB + 10M req CloudFront, 25 GB DynamoDB storage, 100K DPU + 1 GB-mo DSQL, 1M req + 400K GB-s Lambda).

Line	Demo (~0 traffic)	1,000 MAU (0.82M min/mo)	10,000 MAU (8.17M min/mo)
CloudFront egress	\$0	\$0 (0.98 TB ≤ 1 TB free)	\$748 (9.8 TB – 1 free, all in \$0.085 tier)
CloudFront requests	\$0	\$0 (8.2M ≤ 10M free)	\$71.70 (81.7M – 10M free)
DynamoDB writes	~\$0	\$2.04 (3.3M WRU)	\$20.43 (32.7M WRU)
DynamoDB reads + storage	~\$0	\$0.05	\$0.51 (storage ≤ 25 GB free)
Aurora DSQL	\$0 (scale-to-zero)	\$0 (60K DPU ≤ 100K free)	\$3.97 (596K DPU – 100K free)
Lambda rollup	\$0	\$0 (free tier)	\$0 (~82K inv, ~3K GB-s ≤ free tier)
KMS key	\$1	\$1	\$1
S3 catalog storage + origin	~\$0.85	~\$0.90	~\$2.50
Total / month	≈ \$2	≈ \$4	≈ \$848 (≈ \$0.085/user)

The 10K-MAU per-user figure (\$0.085) lands just under the marginal 9.4¢ — the gap is exactly the free-tier subsidy. **Until ~1,000 active users, the entire AWS bill rounds to the \$1/mo KMS key plus catalog storage.**

Fixed / amortized AWS lines (not per-minute)

These don't scale with minutes; they amortize to ~\$0 per stream but are listed for completeness:

Item	Cost	Per-stream impact
S3 audio storage	a 3-min song @ 160 kbps ≈ 3.6 MB × \$0.023/GB-mo = \$0.00008 / song / mo	divided across every stream of that song → negligible
KMS CMK	\$1 / mo per key <i>version</i> ; the stack enables annual rotation, so \$2 / mo from year 2 (AWS caps rotated keys at \$2)	\$0.00000012–24/min amortized (already in the table)
Aurora DSQL / Lambda idle	\$0 — both scale to zero	no standing charge

Item	Cost	Per-stream impact
Bedrock "explain my statement"	~1.5k in × \$1.10/M + 0.4k out × \$5.50/M = \$0.00385 / call (<i>Bedrock on-demand runs ~10% above the direct Anthropic API's \$1/\$5</i>)	only when a user asks; ~+4% to a user's monthly cost if used once

Margin

At an illustrative listener price of **\$0.01/min**:

	Per minute	Per user / mo (817 min)
Listener pays	\$0.01000	\$8.170
All-in AWS cost	\$0.0001155	\$0.0944
AWS as % of price	1.16%	1.16%

AWS consumes ~1% of gross billed revenue. But **AWS is not the dominant infrastructure cost — payment processing is**. Both come out of the same platform take, and Stripe-style card fees run **3–7× the AWS bill**.

What the platform actually bears: AWS *and* inbound payments

The platform's cost is **AWS + the inbound processing fee on wallet top-ups**. Standard card processing is **2.9% + \$0.30** per charge; the fixed \$0.30 makes **top-up frequency** the biggest single lever. Per active listener (gross \$98/yr):

Platform cost line	\$ / user / yr	% of gross
AWS (all-in, marginal)	~\$1.13	~1.2%
Payments — card, monthly top-up	\$6.44	6.6%
Payments — card, annual top-up	\$3.14	3.2%
Payments — ACH-first wallet (0.8%, \$5 cap)	\$0.78	0.8%

Artist payout is *not* a platform cost. The artist earns the royalty (90% of gross); the fee to *disburse* it — Stripe Connect / ACH transfer, ~0.25% of payout volume plus any per-account or instant-payout fees — is deducted from the **artist's** share, not the platform rake. So the platform bears AWS + inbound payments only.

At a 10% platform take (\$9.80/user/yr of rake), the **kept margin after AWS + inbound payments** swings entirely on the rail and top-up size:

Wallet config	Rake kept
Card, monthly top-up	23%

Wallet config	Rake kept
Card, annual top-up	56%
ACH-first, \$10 min / auto-reload	80%
Pass-through (listener pays the fee)	88%

So the earlier "~88% platform margin" was **AWS-only and optimistic**. All-in, the platform keeps **23–88% of the rake** — and at a 6% rake with monthly card top-ups it goes **negative**. The rake floor is set by **payments, not AWS**. The fix is a **prepaid wallet with a \$10 minimum on ACH rails** (bank debit caps the fee), with cards as the instant fallback — see [COST_VISUAL.pdf](#) pp. 1, 6–7.

Cost levers (in order of impact)

1. **Bitrate** — egress is 88% of cost and scales linearly. 160 → 128 kbps ≈ –18% all-in; 96 kbps ≈ –35%. Quality tiers; default mobile lower.
2. **HLS segment length** — CloudFront requests are 9% of cost and scale with segment count. 6s → 10s segments ≈ –40% on the request line.
3. **Edge cache-hit ratio** — long TTLs on immutable audio keep S3 origin GETs and KMS calls near zero; popular catalog is served entirely from the edge.
4. **S3 Bucket Keys** — collapse per-object KMS requests ~99%, pinning KMS at the flat key fee. **Not yet enabled in the stack** (`bucketKeyEnabled` is unset on the audio bucket) — without it, every cache-miss GET is a billed `kms:Decrypt` (e.g. at 10K MAU and a 5% miss rate, ~4M calls ≈ **+\$12/mo**). One-line CDK fix.
5. **Egress volume tiers** — CloudFront drops to \$0.08/GB past 10 TB/mo and as low as \$0.02/GB at PB scale, so the dominant line *deflates* as the business grows.
6. **Per-minute (not per-renewal) ledger** — the rollup dedupes 1.33 renewals/min into one DSQL row. The remaining write-side fat is DynamoDB's 4 WRU/min; dropping the EVT item's GSI1 projection from `ALL` to `KEYS_ONLY` would shave the index write for large events.

Margin levers (bigger than all of the above):

1. **Payment rail** — ACH/bank debit caps the fee (Stripe 0.8% → \$5 ceiling; Adyen \$0.26 flat) vs. cards' uncapped 2.9% + \$0.30. For a wallet, ACH-first ≈ 4× cheaper than cards → +58 pts of kept rake.
2. **Top-up size** — the fixed \$0.30/charge means fewer, larger top-ups win. Monthly → annual card top-ups: \$6.44 → \$3.14/user/yr. Enforce a **\$10 minimum** (keeps even the card fallback under ~6%).
3. **Avoid Stripe metered (per-user monthly invoicing)** — it maxes the fixed-fee count *and* adds the 0.5% Billing fee → 7.1% of gross, the worst option. Keep metering internal (DynamoDB/DSQL); settle aggregates only.

Assumptions & caveats

- **Bitrate 160 kbps / 1.2 MB per minute** and **3.0 min average song** (consistent with 9,800 min/yr ÷ 3,278 songs ≈ 3 min). Both are direct linear scalars — adjust and every figure moves proportionally.
- **All unit rates verified 2026-06-12 via the AWS Pricing API** (us-east-1 on-demand): CloudFront \$0.085/GB + \$0.01/10k HTTPS; DynamoDB \$0.625/M WRU, \$0.125/M RRU, \$0.25/GB-mo; DSQL \$8/M DPU, \$0.33/GB-mo; Lambda \$0.20/M + \$16.67/M GB-s; KMS \$1/key-version-mo + \$0.03/10k; S3 \$0.023/GB-mo + \$0.40/M GET; Bedrock Haiku 4.5 \$1.10/M in, \$5.50/M out.
- **Payment rates are published standard US pricing, not a metered API** (so unlike the AWS lines they aren't programmatically verifiable): card 2.9% + \$0.30; Stripe ACH 0.8% capped \$5; Adyen card ~2.0% + \$0.13 (Interchange++, card-mix dependent); Adyen ACH \$0.26 flat; USDC ~1.5%; Stripe Billing +0.5%; Connect payout ~0.25%. **Volume/Interchange++ deals lower the % at scale; international cards (+1.5%), FX, and disputes (\$15) push it up** — so real-world payments likely run a bit higher than modeled. **Artist payout** (the fee to disburse earned royalties — ~0.25% of payout volume + Connect per-account/instant fees) is borne by the **artist** (deducted from their royalty), not the platform rake, so it sits outside the platform-cost figures above.
- **Write cadence:** `/api/renew` runs every ~45s (60s cookie), and each call writes the balance item and the metered-event item; the event also lands in GS11 (`ALL` projection). Hence ~4 WRU per streamed minute, not 2. The *ledger* stays at 1 row/min because the rollup's idempotency key is per-minute.
- **DynamoDB Streams cost \$0 here:** the only consumer is the rollup Lambda event-source mapping, whose GetRecords calls AWS does not bill. The \$0.02/100k rate applies only to non-Lambda consumers.
- **Lambda invocation count is traffic-density-dependent:** full 100-event batches at scale (~0.01 inv/min), up to ~1 inv/min when events are sparse. Worst case adds ~\$0.0003 per 1,000 min — still <0.3% of total.
- **Aurora DSQL DPU-per-minute is modeled, not measured** (~0.073 DPU/min). The stack isn't deployed yet; validate against `AWS/AuroraDSQL TotalDPU` CloudWatch metrics once the rollup runs. Even at 5× the assumed DPU, DSQL stays under 3% of all-in cost — the conclusion (databases ≈ free per stream) is robust.
- Figures are **marginal** (cost of one more minute). Perpetual free tiers (1 TB CloudFront egress + 10M requests, 25 GB DynamoDB, 100K DSQL DPU + 1 GB-mo, 1M Lambda req + 400K GB-s, 20K KMS requests) make the **first slice of traffic effectively free** — at low volume the real bill rounds to the **\$1/mo KMS key** (see the monthly-bill table).

Appendix — how the rates were verified

All prices pulled live from the AWS Pricing API (us-east-1 endpoint), e.g.:

```
# DynamoDB on-demand / storage / streams
aws pricing get-products --region us-east-1 --service-code AmazonDynamoDB \
```

```

--filters "Type=TERM_MATCH,Field=location,Value=US East (N. Virginia)"

# Aurora DSQL (DPU + storage)
aws pricing get-products --region us-east-1 --service-code AuroraDSQL \
  --filters "Type=TERM_MATCH,Field=location,Value=US East (N. Virginia)"

# CloudFront egress tiers + request pricing
aws pricing get-products --region us-east-1 --service-code AmazonCloudFront \
  --filters "Type=TERM_MATCH,Field=productFamily,Value=Data Transfer" \
    "Type=TERM_MATCH,Field=fromLocation,Value=United States"

# Lambda, S3, KMS – service codes AWSLambda, AmazonS3, awskms (same pattern)

# Bedrock Claude Haiku 4.5 (newer models live under FoundationModels)
aws pricing get-products --region us-east-1 \
  --service-code AmazonBedrockFoundationModels \
  --filters "Type=TERM_MATCH,Field=regionCode,Value=us-east-1"

```

Key dimensions returned (2026-06-12): DDB `WriteRequestUnits $0.625/M`,
`Streams $0.0000002/read-request-unit (free for Lambda pollers)`; DSQL `USE1-DSQL-DistributedProcessingUnits $0.000008/DPU`, `Storage $0.33/GB-Mo`; CloudFront `0-10 TB $0.085/GB`, `HTTPS $0.000001/req`; KMS `$1/key-version-mo`; Bedrock Claude Haiku 4.5: `$1.10/M input`, `$5.50/M output tokens`.